

北 京 交 通 大 学

2021 —2022 学 年 第 一 学 期 期 末 考 试 试 题

课程名称: 数据结构 (A)

出题教师: 黄雅平, 李向前, 孙永奇, 王志海, 徐薇, 原继东

专业: _____ 班级: _____ 姓名: _____ 学号: _____

题号	一	二	三	四	五	六		总分
得分								
阅卷人								

备注: 请把答案写在答题卡上。

一、 单选题(每题 2 分, 共 10 小题, 共 20 分)

- 1、 一个单链表没有头结点, 它的头指针是 L, 通过如下 () 的表示可以指向第二个数据元素。
A. L B. L->next C. L->next->next D. L->next->data
- 2、 一组字符 abcdef 按顺序先进入一个栈, 然后出栈, 再按顺序进入一个队列再出队列, 最后得到字符序列为 ()。
A. abcdef B. aebdcf C. fedcba D. feadbdc
- 3、 已知主串 S='ababcbaa', 模式串 T='ababcaab', 则 next 函数值及 nextval 函数值为 ()。
A. 01123112 和 01013102 B. 01123122 和 00011011
C. 01123112 和 01011001 D. 01123122 和 01013021
- 4、 设数组 data[m]作为循环队列 q 的存储空间, front 为队头指针, rear 为队尾指针, 判定队列 q 为满的条件是 ()。
A. q.front = q.rear B. (q.front+1)% m = q.rear
C. (q.rear+1)% m = q.front D. (q.rear+1)% (m-1) = q.front
- 5、 二叉树中度为 2 的结点有 15 个, 度为 1 的结点有 14 个, 则结点总数为 ()。
A. 29 B. 43 C. 45 D. 46
- 6、 由树转化得到的二叉树称为该树对应的二叉树, 则下面 () 是正确的。

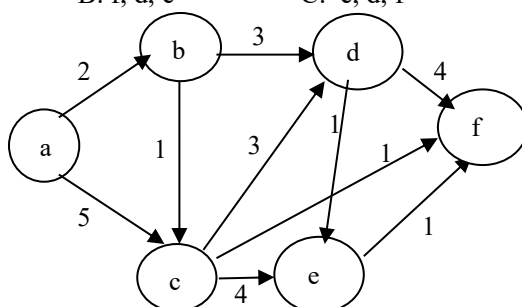
- A. 树的先根遍历序列与其对应的二叉树先序遍历序列相同。
- B. 树的先根遍历序列与其对应的二叉树中序遍历序列相同。
- C. 树的后根遍历序列与其对应的二叉树后序遍历序列相同。
- D. 以上都不对。

7、一棵赫夫曼树共有 69 个结点，对其进行赫夫曼编码，总共得到 () 个不同的编码。

- A. 34
- B. 35
- C. 69
- D. 70

8、如下有向带权图，若采用迪杰斯特拉 (Dijkstra) 算法求源点 a 到其它各顶点的最短路径，得到的第一条最短路径的目标顶点是 b，第二条最短路径的目标顶点是 c，后续得到的其余各最短路径的目标顶点依次是 ()。

- A. d, e, f
- B. f, d, e
- C. e, d, f
- D. f, e, d



9、一棵 3 阶 B-树中含有 1023 个关键字，包含空的叶子结点层，该树的最大深度为 ()。

- A. 9
- B. 10
- C. 11
- D. 12

10、对序列(21,15,32,18,28,6,23)用希尔排序方法排序，经一趟排序后序列变为(21,6,23,18,28,15,32)，则该趟排序采用的增量是 ()。

- A. 2
- B. 3
- C. 4
- D. 5

二、 填空题(每题 1 分，共 15 小题，共 15 分)

11、数据结构的形式定义为 $\langle D, S \rangle$ ，其中 D 是数据元素的有限集，S 是 D 上_____的有限集。

12、若一个算法中的语句频度之和为 $T(n)=500n+n^2$ ，则算法的时间复杂度为_____。

13、在表长为 n 的顺序表中，如果要在第 i ($1 \leq i \leq n$) 个数据元素之后插入一个新的元素，要移动_____个数据元素。

14、对串 $S='BEIJINGJIAOTONG'$ ，执行 $\text{Substring}(\&\text{Sub}, S, 10, 4)$ ，则 $\text{Sub}=______$ 。

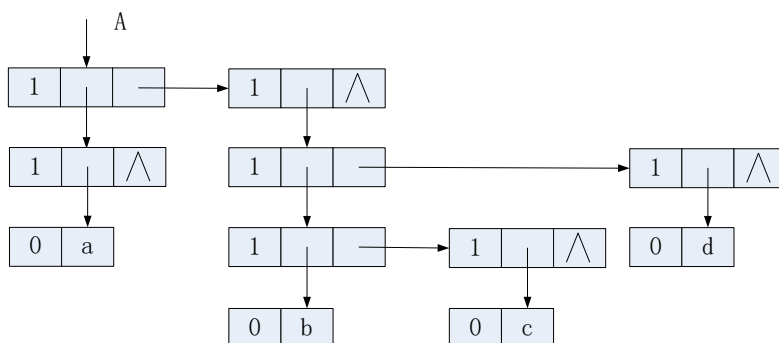
15、稀疏矩阵中的元素以三元组顺序表来表示，则矩阵中的元素(7,5,19)在相应转置矩阵中的表示为_____。

- 16、设森林 F 对应的二叉树为 B，它有 m 个结点，B 的根为 p ， p 的右子树结点个数为 n ，森林 F 中第一棵树的结点个数是_____。
- 17、在赫夫曼树中有 10 个叶子结点，度为 1 的结点有_____个。
- 18、 n 个顶点的无向连通图用邻接矩阵表示时，该矩阵至少有_____个非零元素。
- 19、对于有 e 条边的无向图，所有顶点的度的和为_____。
- 20、_____算法适用于求边比较稠密的网的最小生成树。
- 21、从有序表(17,19,32,43,56,78,80,99)中折半查找元素 80 时，查找成功时的比较次数为_____次。
- 22、一棵深度为 K 的平衡二叉树，所有结点的平衡因子都为 0，则该树共有_____个结点。
- 23、采用链地址法解决冲突的哈希表中，查找成功的平均查找长度只与_____有关。
- 24、冒泡排序在最好情况下的时间复杂度为_____。
- 25、若需要在 $O(n\log n)$ 的时间内完成对数组的排序，且要求占用辅助空间最少，则应选择_____排序方法。

三、 简答题(每题 7 分，共 5 小题，共 35 分)

- 26、有三个元素的进栈顺序为 X、Y、Z，列举出此三个元素可能的出栈序列，并写出相应的进栈和出栈操作序列（假设以 `push` 和 `pop` 分别表示进栈和出栈操作）。

- 27、已知广义表 A 按表头表尾分析法的存储结构如下图所示, 请写出该广义表, 并求其深度和长度。



- 28、(1) 已知一棵二叉树的中序遍历序列为 DBACFEGMKLPNS, 后序遍历序列为 DAFGECBLSNPKM, 画出这棵二叉树, 并给出层次遍历序列。(2) 假设(1)中形态的二叉树是由森林按照孩子兄弟链表表示法转换而来的, 画出森林。

29、已知一组关键字序列为：(35,21,47,60,52,50,73,92)，画出其构成的平衡二叉树，给出主要的平衡过程。

30、已知关键字序列(45, 27, 20, 32, 19, 58, 12, 25, 38, 16)，用哈希函数 $H(\text{key}) = \text{key} \bmod 13$ 构建哈希表，散列地址空间为 0-12，用开放定址法的二次探测再散列方法 $H_i = (H(\text{key}) + d_i) \bmod 13$ ($d_i = 1^2, -1^2, 2^2, -2^2, 3^2, \dots$) 处理冲突，请画出所构造的哈希表，并计算等概率情况下查找成功时的平均查找长度。

四、 算法阅读题（共 2 小题，共 10 分）

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
typedef struct BitNode {  
    int data;  
    struct BitNode * Lchild;  
    struct BitNode * Rchild;  
}*BitTree;
```

```
BitTree F1(BitTree root) {  
    int data ;  
    scanf("%d", &data);  
    if(data == 0) {  
        root = NULL;  
    } else {  
        root = (BitTree)malloc(sizeof(BitNode));  
        root->data= data;  
        if(!root) {  
            printf("Fail!\n");  
            exit(1);  
        }  
        root->Lchild = F1(root->Lchild);  
        root->Rchild = F1(root->Rchild);  
    }  
    return root;  
}
```

```
void F2(BitTree root) {  
    BitTree stack[256];  
    BitTree p = root;  
    int tag[256], top=0;  
    do  
    { while(p!=NULL){
```

```

    top++; stack[top]=p; tag[top]=0; p=p-> Lchild;
}
while((top>0) && tag[top]==1){
    printf("%d ",stack[top]->data); top--;
}
p=stack[top];
if((top>0)&&(tag[top]==0)){
    p=p->Rchild; tag[top]=1;}
}while(p!=NULL || top!=0);
}

```

```

int F3(BitTree root){
    int t1=0,t2=0,out=0;
    if(!root) out=0;
    else{
        t1= F3(root->Lchild);
        t2= F3(root->Rchild);
        out=1+(t1>t2?t1:t2);
    }
    return out;
}

```

```

int main() {
    BitTree root = NULL ;
    root = F1(root);
    F2(root);
    printf("\n");
    printf("%d \n",F3(root));
}

```

31、 (6 分) 说明算法中 F1, F2 和 F3 的功能。

- 32、 (4分) 若 F1 输入为 2 5 6 9 0 0 0 8 0 0 4 1 0 0 3 0 0，其中输入用空格分隔，0 表示 NULL，请写出程序运行结果。

五、 算法填空题（每空 2 分，共 10 分）

下面算法构造图的最小生成树，请把算法中划横线处缺失的语句补齐。

```
struct
{ VertexType adjvex; // U 集中的顶点序号
  VRType lowcost; // 边的权值
} closedge [MAX_VERTEX_NUM];

void MiniSpanTree_PRIM( MGraph G, VertexType u )
{ //用普里姆算法从顶点 u 出发构造网 G 的最小生成树
  _____ ( 33 ) _____ = LocateVex ( G, u );
  for ( j = 0; j < G.vexnum; ++j ) // 辅助数组初始化
    if ( j _____ ( 34 ) _____ k )
      closedge[j] = { u, G.arcs[k][j].adj }; // {adjvex, lowcost}
  closedge[k].lowcost = 0; // 初始，U = {u}
  for ( i = 1; i < G.vexnum; ++i ) //选择其余 N-1 个顶点
  {
    k = minimum(closedge); // 求出加入生成树的下一个顶点 k
    printf(closedge[k].adjvex, G.vexs[k]); // 输出生成树上一条边的两个顶点
    closedge[k].lowcost = _____ ( 35 ) _____; // 第 k 顶点并入 U 集
    for ( j = 0; j < G.vexnum; ++j ) // 修改其它顶点的最小边
      if ( G.arcs[k][j].adj _____ ( 36 ) _____ closedge[j].lowcost )
        _____ ( 37 ) _____ = { G.vexs[k], G.arcs[k][j].adj };
  }
}
```


六、 算法设计题（共 1 题，共 10 分）

38、设二维数组为 $\text{int } A[n][n]$ ($n < 500$)，试对该二维数组设计一个排序算法

$\text{ArraySort}(\text{int } A[500][500], \text{int } n),$

排序规则如下：

根据二维数组中每一行数字的和，用快速排序算法对行进行非递增排序。排序示例如下图所示，其中(a)为原始的二维数组，(b)为执行排序规则后的结果。

7 2 8 3	→	9 7 1 8
9 7 1 8		6 7 5 6
4 3 6 5		7 2 8 3
6 7 5 6		4 3 6 5
(a)		(b)

北 京 交 通 大 学

2021 —2022 学年第 一 学期期末考试试题标准答案

课程名称: 数据结构 (A)

出题教师: 黄雅平, 李向前, 孙永奇, 王志海, 徐薇, 原继东

一、单选题(每题 2 分, 共 10 小题, 共 20 分)

1~10: BCDCC ABBCC

二、填空题 (每题 1 分, 共 15 小题, 共 15 分)

- 11、关系
- 12、 $O(n^2)$
- 13、 $n-i$
- 14、‘AOTO’
- 15、(5,7,19)
- 16、 $m-n$
- 17、0
- 18、 $2(n-1)$
- 19、 $2e$
- 20、Prim 或普里姆
- 21、3
- 22、 2^k-1
- 23、装填因子或 α
- 24、 $O(n)$
- 25、堆

判分标准: 有微小错误可给 0.5 分。

三、简答题(每题 7 分, 共 5 小题, 共 35 分)

26、判分标准: 错一个扣 1.5 分, 扣完 7 分为止。

答案:

出栈顺序

X、Y、Z
X、Z、Y
Y、X、Z
Y、Z、X
Z、Y、X

操作序列

push pop push pop push pop
push pop push push pop pop
push push pop pop push pop
push push pop push pop pop
push push push pop pop pop

27、判分标准: 广义表 3 分, 长度 2 分, 深度 2 分。

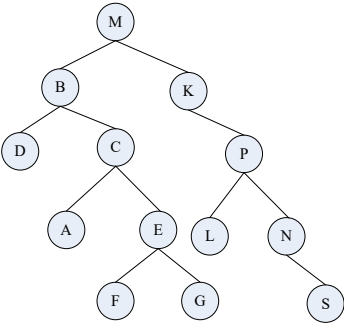
答案: 广义表 $((a), ((b,c),d))$;

长度 2 ;

深度 3

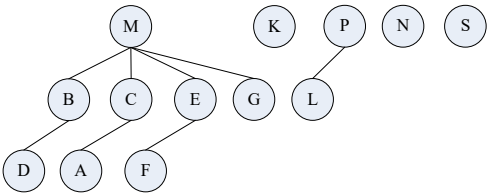
28、判分标准: 二叉树 3 分, 层序 1 分, 森林 3 分。

答案： (1)二叉树如下图所示



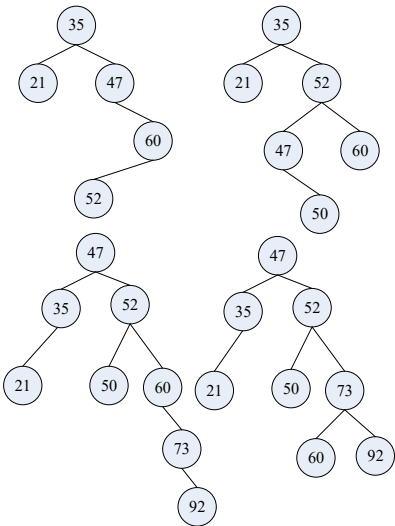
层次遍历序列：MBKDCPAELNFGS

(2)森林如下图所示



29、判分标准：错一个扣 2 分，扣完为止。

答案：



30、判分标准：哈希表全对给 5 分，其中每个关键字位置给 0.5 分；平均查找长度 2 分。

答案：

地址空间	0	1	2	3	4	5	6	7	8	9	10	11	12
关键字	25	27	58	16		32	45	20			19	38	12
比较次数	2	1	5	1		3	1	1			4	3	1

查找成功时的平均查找长度 $ASL = (2+1+5+1+3+1+1+4+3+1)/10 = 2.2$

四、算法阅读题（共 2 小题，共 10 分）

31、F1: 递归创建一个二叉树 （2 分）

F2: 后序非递归遍历二叉树 （2 分）

F3: 递归求二叉树深度 （2 分）

32、程序输出结果：

96851342 （2 分）

4 （2 分）

五、算法填空题（每空 2 分，共 10 分）

33、k

34、!=

35、0

36、<

37、closedge[j]

六、算法设计题（共 1 题，共 10 分）

38、

```
void CopyArray(int a[], int b[], int n)
```

```
{
    for (int i=1; i<=n; i++) {
        a[i] = b[i];
    }
}
```

//一趟快速排序

```
int Partition(int S[], int low, int high, int A[][500], int n)
```

```
{
    int pivotkey;
    S[0] = S[low];
    CopyArray(A[0], A[low], n);
    pivotkey = S[low];
    while (low < high) { (1 分)
        while (low < high && S[high] <= pivotkey) { (1 分)
            --high;
        }
        S[low] = S[high]; (1 分)
        CopyArray(A[low], A[high], n); (2 分)
        while (low < high && S[low] >= pivotkey) { (1 分)
            ++low;
        }
        S[high] = S[low];
        CopyArray(A[high], A[low], n);
    }
    S[low] = S[0];
    CopyArray(A[low], A[0], n);

    return low;
}
```

```
}
```

//快速排序

```
void QSort(int S[], int low, int high, int A[][500], int n)
```

```
{
```

```
    int pivotloc;
```

```
    if (low < high) {
```

```
        pivotloc = Partition(S, low, high, A, n);    (1 分)
```

```
        QSort(S, low, pivotloc - 1, A, n);    (1 分)
```

```
        QSort(S, pivotloc + 1, high, A, n);
```

```
    }
```

```
}
```

//对二维数组的排序算法

```
void ArraySort(int A[500][500], int n)
```

```
{
```

//对二维数组中每一行数字求和并根据二维数组中每一行数字的和，用快速排序算法对行进行非递增排序

```
    int S[500];
```

```
    for (int i=1; i<=n; i++) {    (2 分)
```

```
        S[i] = 0;
```

```
        for (int j=1; j<=n; j++) {
```

```
            S[i] += A[i][j];
```

```
        }
```

```
    }
```

```
    QSort(S, 1, n, A, n);
```

```
}
```

判分标准：

快速排序 6 分，数组移动 2 分，排序函数 ArraySort 2 分。